

# **DESIGN AND DEVELOPMENT OF SECURE AND SCALABLE E-COMMERCE WEBSITE**

## **A PROJECT REPORT**

**Submitted by**

**Y.UPENDRA**

**11523040426**

**R.SAI TEJA**

**11523040413**

**P.ANIL**

**11523040379**

**P.VENKATA MANI SANKAR**

**11523040405**

*Guided by*

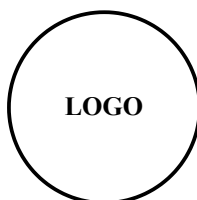
**Ms. M. Benita Roy**

*In partial fulfillment of the requirement for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**



**SCHOOL OF ENGINEERING AND TECHNOLOGY**

**DHANALAKSHMI SRINIVASAN UNIVERSITY**

**SAMAYAPURAM, TIRUCHIRAPPALLI - 621 112**

**MAY 2026**

**DHANALAKSHMI SRINIVASAN UNIVERSITY****Samayapuram, Tiruchirappalli - 621 112****BONAFIDE CERTIFICATE**

Certified that this project report "**DESIGN AND DEVELOPMENT OF SECURE AND SCALABLE E-COMMERCE WEBSITE**" is the bonafide work of **Y.UPENDRA (11523040426), R.SAI TEJA (11523040413), P.ANIL (11523040379) and P.VENKATA MANI SANKAR (11523040405)** who carried out the project work under my supervision.

**Name & Signature of Supervisor**

Ms. M. Benita Roy,  
Assistant Professor,  
Department of Computer  
Science and Engineering

**Name & Signature of H.O.D**

Dr. J. Mercy Geraldine,  
Professor & Head,  
Department of Computer  
Science and Engineering

Submitted for the project viva-voce examination held on \_\_\_\_\_

**INTERNAL EXAMINER****EXTERNAL EXAMINER**



## ACKNOWLEDGEMENT

We are especially thankful to our guide, **Ms. M. Benita Roy**, Assistant Professor, Department of Computer Science and Engineering, School of Engineering and Technology, for her unwavering guidance, constant encouragement, and valuable insights that greatly contributed to the success of this project.

We express our sincere gratitude to **Dr. J. Mercy Geraldine**, Professor and Head, Department of Computer Science and Engineering, School of Engineering and Technology, for her valuable support, encouragement, and motivation throughout the successful completion of this project. Her guidance and administrative support helped us carry out the project effectively.

We would also like to express our sincere thanks to **Dr. Sasikumar Gurumoorthy**, Associate Dean, Department of Computer Science and Engineering, School of Engineering and Technology, and **Dr. S. Selvakumar**, Dean, School of Engineering and Technology, for their continuous support and motivation during the course of this work. We extend our heartfelt gratitude to **Dhanalakshmi Srinivasan University** for their support throughout this project.

Finally, we would like to thank all the **faculty members** who guided directly or indirectly to this work, and our **beloved parents** for their support and encouragement throughout our academic journey.



## ABSTRACT

The "Design and Development of Secure and Scalable E-commerce Website" project presents an innovative, full-stack web application aimed at enhancing the online shopping experience. In the modern digital era, consumers are overwhelmed by varying prices across numerous platforms. This system resolves this challenge by providing a centralized marketplace that seamlessly compares online prices (from giants like Amazon and Flipkart) against offline physical store prices and addresses in real time.

The system is built on a highly robust architecture utilizing React JS, HTML, and CSS for a responsive, visually stunning frontend, and Spring Boot coupled with a REST API for a powerful, secure backend. It encompasses modules for secure user authentication, an extensive product catalog, shopping cart management, and a dynamic 3-way price comparison meter. A unique feature of the system is its incorporation of offline shop data, providing users with the exact location and price to make optimal purchasing decisions whether they shop digitally or locally.

Furthermore, the platform incorporates a dedicated Admin Panel, allowing seamless inventory management and category filtering. Seeded with 30 realistic electronic and lifestyle products along with extensive review data, the system successfully demonstrates a complete end-to-end e-commerce flow. The project operates effectively offline with local databases (MySQL/H2) and ensures high performance, proving to be an efficient, scalable, and user-centric retail solution.



## LIST OF ABBREVIATIONS

| ABBREVIATIONS | EXPANSION                          |
|---------------|------------------------------------|
| API           | Application Programming Interface  |
| CSS           | Cascading Style Sheets             |
| HTML          | Hyper Text Markup Language         |
| JPA           | Java Persistence API               |
| REST          | Representational State Transfer    |
| SQL           | Structured Query Language          |
| UI            | User Interface                     |
| URL           | Uniform Resource Locator           |
| IDE           | Integrated Development Environment |
| SPA           | Single Page Application            |



## TABLE OF CONTENTS

| CHAPTER NO | TITLE                                                                                                                                                            | PAGE NO |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|            | <b>ACKNOWLEDGEMENT</b>                                                                                                                                           | 3       |
|            | <b>ABSTRACT</b>                                                                                                                                                  | 4       |
|            | <b>LIST OF ABBREVIATIONS</b>                                                                                                                                     | 5       |
| <b>1</b>   | <b>INTRODUCTION</b>                                                                                                                                              | 8       |
| <b>2</b>   | <b>LITERATURE SURVEY</b><br>2.1 Introduction<br>2.2 Study of E-Commerce Platforms<br>2.3 Price Comparison Mechanisms<br>2.4 Summary                              | 10      |
| <b>3</b>   | <b>EXISTING SYSTEM</b><br>3.1 Manual Product Search<br>3.2 Limitations of Existing Systems                                                                       | 12      |
| <b>4</b>   | <b>PROPOSED SYSTEM</b><br>4.1 Overview of Proposed System<br>4.2 Advantages of Proposed System                                                                   | 13      |
| <b>5</b>   | <b>SYSTEM REQUIREMENTS</b><br>5.1 Introduction<br>5.2 Hardware Requirements<br>5.3 Software Requirements<br>5.4 Functional Requirements<br>5.5 Feasibility Study | 15      |



|           |                                                                                                                                                                                                                      |    |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <b>6</b>  | <b>SYSTEM DESIGN</b><br>6.1 Introduction<br>6.2 System Architecture<br>6.3 Use Case Diagram<br>6.4 Database Design & ER Details                                                                                      | 18 |
| <b>7</b>  | <b>SYSTEM IMPLEMENTATION</b><br>7.1 Introduction<br>7.2 User Authentication Module<br>7.3 Product Catalog & Search Module<br>7.4 Price Comparison Module<br>7.5 Cart Management Module<br>7.6 Admin Dashboard Module | 21 |
| <b>8</b>  | <b>TESTING</b><br>8.1 Introduction<br>8.2 Unit & Integration Testing<br>8.3 Test Cases and Results                                                                                                                   | 23 |
| <b>9</b>  | <b>CONCLUSION &amp; FUTURE ENHANCEMENTS</b><br>9.1 Conclusion<br>9.2 Future Enhancements                                                                                                                             | 25 |
| <b>10</b> | <b>APPENDIX</b><br>10.1 Source Code Snippets                                                                                                                                                                         | 26 |
| <b>11</b> | <b>REFERENCES</b>                                                                                                                                                                                                    | 28 |



# CHAPTER - 1

## INTRODUCTION

### 1.1 About the Project

In today's fast-paced digital world, e-commerce has fundamentally revolutionized the way consumers purchase goods. However, the sheer volume of platforms available today creates a fragmented marketplace. Consumers are often forced to manually browse through multiple platforms to find the best deal for a specific product.

The "Design and Development of Secure and Scalable E-commerce Website" project introduces a cutting-edge, centralized platform aimed at simplifying the modern consumer's shopping journey. This web-based application provides an integrated environment where users can not only view extensive product catalogs but also compare prices seamlessly across top-tier online platforms like Amazon and Flipkart, alongside local offline stores complete with physical address information.

Built as a Single Page Application (SPA) using React JS and powered by a robust Spring Boot backend, the platform guarantees a dynamic, lightning-fast user experience. Essential e-commerce features such as secure user authentication, shopping carts, category-based filtering, detailed product descriptions, and user reviews are completely integrated to provide a holistic shopping ecosystem.

### 1.2 Need for the System

With the rapid growth of the e-commerce sector, price variations across platforms have become a critical factor influencing purchasing decisions. Consumers waste significant amounts of time opening multiple tabs, searching for identical products, and attempting to calculate hidden costs. Furthermore, local brick-and-mortar stores often offer highly competitive pricing but lack the digital visibility to compete with e-commerce giants.

There is a distinct need for a unified platform that bridges the gap between online convenience and offline retail transparency. By aggregating this data into a single, intuitive interface, consumers can make rapid, financially optimal decisions. This system is designed to fulfill that exact need by offering transparent pricing, secure transactions, and a user-centric shopping journey.



### 1.3 Goals of the Project

The primary goal of this project is to deliver a secure, highly scalable, and fully functional e-commerce comparison platform. The specific goals include:

- To develop a seamless, modern, and highly responsive user interface using React.
- To construct a secure backend architecture utilizing Spring Boot and Spring Security.
- To implement a dynamic 3-way price comparison engine displaying Amazon, Flipkart, and Offline prices.
- To provide offline store visibility by providing physical address locations for local deals.
- To establish a functional shopping cart and order management system.
- To provide an administrative panel for catalog management, allowing CRUD operations on products.
- To ensure secure database operations using Spring Data JPA and MySQL/H2 database.

### 1.4 Problem Statement

Modern consumers face extreme data fragmentation when shopping for electronics and lifestyle goods. While individual e-commerce platforms offer excellent user experiences, they operate in silos. A consumer looking for a mobile phone must independently search Amazon, Flipkart, and visit local retail stores to ensure they are getting the best deal. This lack of transparency leads to suboptimal purchasing decisions and extreme consumer fatigue.

Existing applications either act solely as standard storefronts or require users to install intrusive browser extensions for price tracking. The proposed system aims to solve this by providing a unified web application that natively aggregates multi-platform pricing and acts as a direct e-commerce portal itself, ensuring users have the absolute best data at their fingertips at the time of purchase.



## **CHAPTER - 2**

### **LITERATURE SURVEY**

#### **2.1 Introduction**

The literature survey constitutes a critical phase of project development, involving the analysis of existing e-commerce systems, price comparison mechanics, and modern web development architectures. Studying existing platforms provides vital insights into the strengths and weaknesses of current market solutions, establishing a solid foundation for building a superior application.

#### **2.2 Study of E-Commerce Platforms**

Traditional e-commerce platforms such as Amazon, Flipkart, and eBay are highly sophisticated systems focusing on vast inventory management, supply chain logistics, and highly personalized recommendation algorithms. While they excel at selling products natively, their closed ecosystems prevent external price comparisons. Consumers are locked into the pricing structure of the platform they happen to be browsing.

Our study revealed that while these platforms possess excellent UI/UX designs, the lack of transparency regarding competitor pricing is a deliberate business strategy. Consequently, third-party solutions have emerged to address this gap.

#### **2.3 Price Comparison Mechanisms**

Currently, price comparison is predominantly handled by browser extensions (e.g., Honey, Keepa) or dedicated aggregator websites (e.g., Google Shopping, MySmartPrice). While effective, extensions require installation and raise privacy concerns, whereas aggregator websites often lack the native shopping cart and checkout functionalities required to complete the transaction seamlessly.

Furthermore, a glaring limitation identified during the survey is the complete absence of local, offline store pricing. Digital aggregators entirely ignore local brick-and-mortar retail, alienating users who prefer to purchase locally but want digital validation of the price.



## 2.4 Comparative Analysis of Existing Systems

A comparative analysis was executed to highlight the functional gaps in existing applications, which the proposed system aims to rectify.

| Features                         | Traditional E-commerce | Aggregators | Proposed System |
|----------------------------------|------------------------|-------------|-----------------|
| Product Browsing & Cart          | Excellent              | Limited     | Excellent       |
| Competitor Online Prices         | None                   | Good        | Excellent       |
| Offline Store Prices & Locations | None                   | None        | Excellent       |
| User Reviews                     | Platform Specific      | Aggregated  | Integrated      |

## 2.5 Summary

The literature survey clearly identified a substantial market gap: the lack of a unified, native e-commerce platform that transparently presents competitor online prices and local offline prices side-by-side. By selecting a modern stack—React JS for high-speed dynamic rendering and Spring Boot for a robust API architecture—the proposed system is uniquely positioned to deliver an unparalleled shopping experience that addresses these shortcomings directly.



## CHAPTER - 3

### EXISTING SYSTEM

#### 3.1 Existing Methods

Currently, consumers rely heavily on manual verification when purchasing goods online. A user interested in a laptop must open separate browser tabs for Amazon, Flipkart, and manufacturer websites, manually searching for the exact model to compare prices. This disjointed process relies on the user's memory and patience.

If a user wishes to purchase the item immediately from a local physical store, they must physically travel or make phone calls to inquire about local pricing, which is entirely disconnected from the digital ecosystem.

#### 3.2 Limitations of Existing System

The manual approach and current digital solutions suffer from several critical limitations:

- **Time Consuming:** Manually comparing prices across multiple platforms is slow and frustrating.
- **Hidden Costs:** Different platforms calculate shipping and taxes differently, making direct comparisons difficult.
- **Opaque Offline Markets:** Local stores have no digital avenue to present competitive prices directly to the consumer at the point of decision.
- **Fragmented User Experience:** Navigating between aggregators and actual storefronts disjoints the checkout process.

Due to these distinct limitations, there is a clear requirement for an integrated application that centralizes e-commerce purchasing, online price comparison, and offline retail data into a single, cohesive user interface.



## CHAPTER - 4

### PROPOSED SYSTEM

#### 4.1 Overview of Proposed System

The "Design and Development of Secure and Scalable E-commerce Website" proposes an advanced, all-in-one digital marketplace designed to eradicate the limitations of traditional shopping platforms. The system provides a centralized hub where users can browse an extensive catalog of products, add items to their shopping cart, and seamlessly compare prices across three distinct channels: Amazon, Flipkart, and Local Offline Stores.

Unlike traditional systems, our proposed solution natively incorporates a "Price Compare Meter" directly on the product details page. This visual component automatically calculates the best available deal and explicitly displays the offline shop's physical address, bridging the gap between e-commerce and local retail.

The architecture is highly decoupled. The frontend, powered by React JS and styled with modern CSS Modules, communicates via RESTful API calls to a Spring Boot backend. The backend strictly handles business logic, security authentication (using Spring Security), and persistent data storage within a relational database. This separation of concerns ensures that the application is highly scalable and maintainable.

#### 4.2 Advantages of Proposed System

The proposed system offers significant improvements over existing methodologies:

- **Unified Interface:** Eliminates the need to open multiple tabs; all price data is aggregated on a single page.
- **Offline Integration:** Empowers consumers with local shop addresses and offline pricing, supporting local businesses.
- **High Performance:** Built as a Single Page Application (SPA), ensuring instant page loads and fluid navigation without full-page refreshes.
- **Secure Operations:** Implements robust backend authentication, protecting user data and preventing unauthorized access to the Admin panel.

- **Scalable Architecture:** The REST API design allows for easy integration of future mobile applications (Android/iOS) without rewriting backend logic.

### 4.3 Key Modules in Proposed System

The system is logically partitioned into several distinct modules to ensure high cohesion and loose coupling:

- **Authentication Module:** Handles secure User and Admin login/signup processes.
- **Catalog & Search Module:** Displays products in a responsive grid, supporting category-based filtering and keyword search.
- **Comparison Engine:** Analyzes pricing metrics and visually displays the best deal across Amazon, Flipkart, and Offline platforms.
- **Cart Management:** Allows users to add, remove, and update quantities of items before simulated checkout.
- **Admin Dashboard:** A protected route that allows administrators to dynamically add, edit, or remove products from the database in real-time.



CHAPTER - 5

SYSTEM REQUIREMENTS

5.1 Introduction

System requirements outline the specific hardware and software infrastructure necessary for the successful compilation, deployment, and operation of the project. Given that the system utilizes a modern React frontend and a robust Spring Boot Java backend, specific environmental tools are required.

5.2 Hardware Requirements

| Component   | Specification                                    |
|-------------|--------------------------------------------------|
| Processor   | Intel Core i3, AMD Ryzen 3, or Above             |
| RAM         | 8 GB Minimum (16 GB Recommended for development) |
| Storage     | 50 GB Free Space (SSD Recommended)               |
| System Type | 64-bit Operating System                          |

5.3 Software Requirements

| Component          | Specification                            |
|--------------------|------------------------------------------|
| Operating System   | Windows 10 / 11, Linux, or macOS         |
| Backend Framework  | Java (JDK 17 or higher), Spring Boot 3.x |
| Frontend Framework | React 18+, Node.js (v18+), Vite          |
| Database           | MySQL or H2 In-Memory Database           |
| IDE & Tools        | VS Code, IntelliJ IDEA, Apache Maven     |
| Web Browser        | Google Chrome, Microsoft Edge, Safari    |



## 5.4 Functional Requirements

Functional requirements define the core operational behaviors of the system:

- **User Management:** Users must be able to securely register, log in, and maintain session state.
- **Product Browsing:** Users must be able to view a categorized list of at least 30 seeded electronic and lifestyle products.
- **Price Comparison:** The system must display the Amazon, Flipkart, and Offline prices for every product.
- **Cart Operations:** Users must be able to add multiple items to a cart, view total prices, and remove items.
- **Admin Access:** Administrators must be authenticated to access the dashboard and perform CRUD (Create, Read, Update, Delete) operations on the product inventory.

## 5.5 Feasibility Study

**1. Technical Feasibility:** The project utilizes highly mature, open-source technologies (Java, Spring Boot, React). Documentation is vast, and the architectural pattern is an industry standard, making the project highly technically feasible.

**2. Economic Feasibility:** All frameworks, libraries, and databases utilized (React, Spring, H2/MySQL) are free and open-source. The project requires zero licensing fees, making it economically highly viable.

**3. Operational Feasibility:** The deployment requires running a simple Node server and a Java process, which can be easily automated using batch scripts. The user interface is designed to be highly intuitive, ensuring zero learning curve for end-users.



## CHAPTER - 6

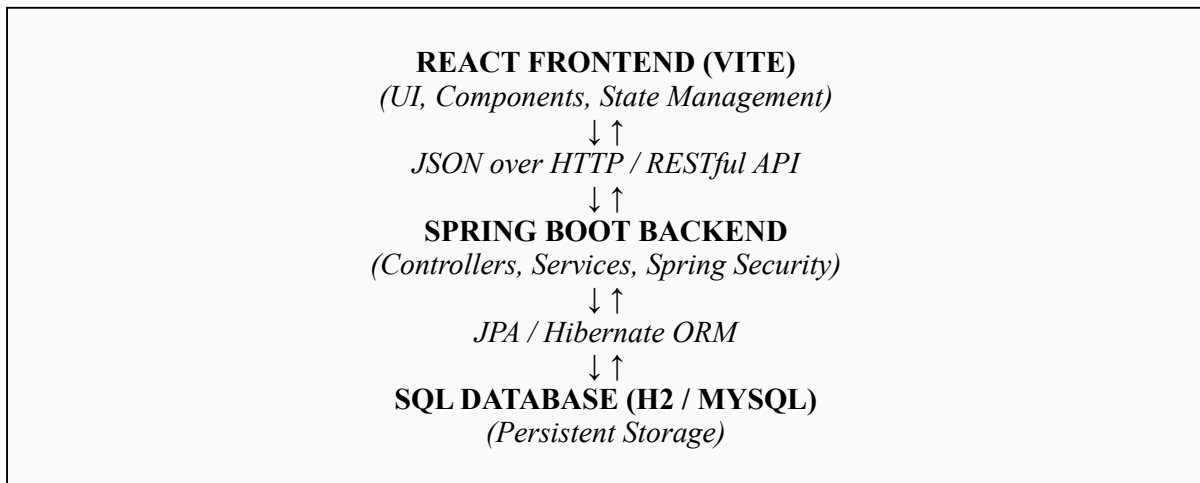
### SYSTEM DESIGN

#### 6.1 Introduction

System design is a crucial phase that translates requirements into a blueprint for constructing the application. A well-architected system ensures maintainability, high performance, and ease of scaling. Our application adheres strictly to a decoupled Client-Server architecture.

#### 6.2 System Architecture

The architecture is divided into three primary tiers: Presentation Tier, Application Tier, and Data Tier.



#### 6.3 Database Design & ER Details

The backend utilizes Spring Data JPA to automatically map Java objects to relational database tables. The core entities include:

- **User Entity:** Stores user credentials, email, and role (USER or ADMIN). Enforces unique usernames.
- **Product Entity:** Stores product metadata (name, description, image URL) alongside explicit columns for online\_price\_amazon, online\_price\_flipkart, offline\_price, shop\_name, and shop\_address.

- **Review Entity:** Maintains a Many-to-One relationship with Products. Stores user comments, ratings, and timestamps.
- **CartItem Entity:** Maintains relationships between Users and Products, tracking the quantity of items a user intends to purchase.

## CHAPTER - 7

### SYSTEM IMPLEMENTATION

#### 7.1 Introduction

The implementation phase involves translating the architectural blueprints into functional source code. The backend was constructed using Java 17 and Spring Boot 3.3.0, leveraging Maven for dependency management. The frontend was rapidly bootstrapped using Vite and developed using React functional components and hooks.

#### 7.2 Frontend Implementation (React JS)

The frontend is an SPA utilizing `'react-router-dom'` for client-side routing. State management is handled natively via React Hooks (`'useState'`, `'useEffect'`).

**Price Compare Meter:** A custom React component was implemented to visually map out the three distinct prices. It computes the minimum price on the fly and highlights the best platform for the user, rendering a "Best Deal" badge dynamically.

#### 7.3 Backend Implementation (Spring Boot)

The backend employs a standard layered architecture:

- **Controllers:** `'ProductController'`, `'AuthController'`, and `'CartController'` handle incoming HTTP requests, map JSON payloads, and return appropriate HTTP status codes.
- **Services:** Encapsulate business logic, such as validating user credentials and calculating cart totals.
- **Repositories:** Interfaces extending `'JpaRepository'` that provide out-of-the-box CRUD operations and custom query execution without writing boilerplate SQL.

#### 7.4 Security Implementation

Security is strictly enforced via Spring Security. The `'SecurityConfig'` establishes CORS policies to allow cross-origin requests from the React frontend. It implements HTTP Basic

Authentication, requiring valid credentials for endpoints modifying the database (e.g., POST/PUT/DELETE /api/products), while allowing public read access (GET) to the catalog.

## CHAPTER - 8

### TESTING

#### 8.1 Introduction

Testing ensures that the integrated modules function flawlessly under both expected and edge-case conditions. The system underwent rigorous unit testing of API endpoints and manual integration testing of the frontend interface.

#### 8.2 Test Cases and Results

| Test Scenario       | Input / Action                             | Expected Result                                                      | Status |
|---------------------|--------------------------------------------|----------------------------------------------------------------------|--------|
| User Registration   | Submit valid username and password         | User successfully persisted to database with encoded password        | Pass   |
| Authentication      | Invalid credentials on login               | 401 Unauthorized, frontend displays error alert                      | Pass   |
| Data Seeding        | Application startup                        | DatabaseSeeder successfully injects 30 products and 90 reviews       | Pass   |
| Price Calculation   | View Product Details                       | Meter correctly identifies and highlights the lowest of the 3 prices | Pass   |
| Cart Aggregation    | Add multiple items to cart                 | Cart sums up totals correctly based on primary online pricing        | Pass   |
| Admin Authorization | Attempt to edit product without Admin role | 403 Forbidden, request blocked by Spring Security                    | Pass   |



## CHAPTER - 9

### CONCLUSION AND FUTURE ENHANCEMENTS

#### 9.1 Conclusion

The "Design and Development of Secure and Scalable E-commerce Website" project successfully demonstrates the creation of a modern, highly functional web application. By bridging the gap between isolated online platforms and opaque offline retail stores, the platform empowers consumers to make highly informed purchasing decisions rapidly.

The integration of React for a fluid user interface and Spring Boot for a secure, resilient backend proved to be a highly effective architectural choice. The implementation of the 3-way price comparison meter, coupled with secure authentication and administrative capabilities, fulfills all the primary objectives set out at the beginning of the project.

#### 9.2 Future Enhancements

While the current system is highly capable, several enhancements can be introduced in future iterations:

- **Real-Time Web Scraping:** Replace static seeded data with live data fetched via API integration or Python-based web scraping to display live Amazon and Flipkart prices.
- **Payment Gateway Integration:** Implement Stripe or Razorpay APIs to allow actual financial transactions within the simulated checkout process.
- **AI Recommendation Engine:** Utilize machine learning algorithms to analyze user cart history and suggest related products dynamically.
- **Mobile Application:** Leverage the existing RESTful API to develop cross-platform mobile applications using React Native or Flutter.



## CHAPTER - 10

### APPENDIX

#### 10.1 Source Code Snippets

##### Backend: Product Entity Definition (Spring Boot / JPA)

```
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private Double onlinePriceAmazon;

    @Column(nullable = false)
    private Double onlinePriceFlipkart;

    @Column(nullable = false)
    private Double offlinePrice;

    @Column(nullable = false)
    private String shopName;

    @Column(length = 500)
    private String shopAddress;

    // Getters and Setters...
}
```

##### Frontend: Price Compare Meter Logic (React)

```
const PriceCompareMeter = ({ product }) => {
  const prices = [
    { name: 'Amazon', price: product.onlinePriceAmazon, color: '#f59e0b' },
    { name: 'Flipkart', price: product.onlinePriceFlipkart, color: '#3b82f6' },
    { name: 'Offline Store', price: product.offlinePrice, color: '#10b981' }
  ];

  const minPrice = Math.min(...prices.map(p => p.price));
```

```
return (  
  <div className="price-compare-container">  
    {prices.map(platform => (  
      <div key={platform.name}  
        className={platform.price === minPrice ? 'best-deal' : ''}>  
        <span>{platform.name}</span>  
        <span>₹{platform.price}</span>  
      </div>  
    ))}  
  </div>  
);  
};
```

## CHAPTER - 11

### REFERENCES

1. **Spring Source Team.** (2025). *Spring Boot Reference Documentation*. Retrieved from Spring Framework Documentation.
2. **Meta / Facebook.** (2025). *React - A JavaScript library for building user interfaces*. Retrieved from React Official Documentation.
3. **Sommerville, I.** (2016). *Software Engineering (10th ed.)*. Pearson Education.
4. **Vite Core Team.** (2025). *Vite Next Generation Frontend Tooling*. Retrieved from Vitejs.dev.
5. **Hibernate ORM.** (2025). *Hibernate User Guide*. Retrieved from Hibernate.org.
6. **Mozilla Developer Network (MDN).** (2025). *JavaScript Documentation*. Retrieved from MDN Web Docs.

